Ulster University

Cookies/LocalStorage

cp.shewell@ulster.ac.uk

Cookies

- Cookies are small strings of data that are stored directly in the browser. They are a part of HTTP protocol, defined by RFC 6265 specification.
- Cookies are usually set by a web-server using response Set-Cookie HTTP-header.
- Then the browser automatically adds them to (almost) every request to the same domain using Cookie HTTP-header.

Cookies

- One of the most widespread use cases is authentication:
  - Upon sign in, the server uses Set-Cookie HTTP-header in the response to set a cookie with a unique “session identifier”.
  - Next time when the request is set to the same domain, the browser sends the cookie over the net using Cookie HTTP-header.
  - So the server knows who made the request.
- We can also access cookies from the browser, using document.cookie property.

Cookies

- The value of document.cookie consists of name=value pairs, delimited by ;. Each one is a separate cookie.
- To find a particular cookie, we can split document.cookie by ;, and then find the right name.
- We can use either a regular expression or array functions to do that.

Cookies

Writing

- We can write to document.cookie. But it’s not a data property, it’s an accessor (getter/setter). An assignment to it is treated specially.
- **A write operation to document.cookie updates only cookies mentioned in it but doesn’t touch other cookies.**
- For instance, this call sets a cookie with the name user and value John:

```
document.cookie = "user=John"; // update only cookie named 'user'
alert(document.cookie); // show all cookies
```

Cookies

Writing

- If you run it, then probably you'll see multiple cookies. That's because document.cookie= operation does not overwrite all cookies. It only sets the mentioned cookie user.

- Technically, name and value can have any characters, to keep the valid formatting they should be escaped using a built-in encodeURIComponent function:

```
let name = "my name";
let value = "John Smith"

// encodes the cookie as my%20name=John%20Smith
document.cookie = encodeURIComponent(name) + '=' + encodeURIComponent(value);

alert(document.cookie); // ...; my%20name=John%20Smith
```

Cookies

Limitations

- There are few limitations:
  - The name=value pair, after encodeURIComponent, should not exceed 4KB. So we can’t store anything huge in a cookie.
  - The total number of cookies per domain is limited to around 20+, the exact limit depends on a browser.
- Cookies have several options, many of them are important and should be set.
- The options are listed after key=value, delimited by ;

Options

Path

- **path=/mypath**
- The url path prefix, the cookie will be accessible for pages under that path. Must be absolute. By default, it’s the current path.
- If a cookie is set with path=/admin, it’s visible at pages /admin and /admin/something, but not at /home or /adminpage.
- Usually, we should set path to the root: path=/ to make the cookie accessible from all website pages.

Options

Domain

- **domain=site.com**
- A domain where the cookie is accessible. In practice though, there are limitations. We can’t set any domain.
- By default, a cookie is accessible only at the domain that set it. So, if the cookie was set by site.com, we won’t get it other.com.
- …But what’s more tricky, we also won’t get the cookie at a subdomain forum.site.com

Options

Domain

- …But what's trickier, we also won't get the cookie at a subdomain forum.site.com

```
// at site.com
document.cookie = "user=John"

// at forum.site.com
alert(document.cookie); // no user
```

- **There's no way to let a cookie be accessible from another 2nd-level domain, so other.com will never receive a cookie set at site.com.**

- It's a safety restriction, to allow us to store sensitive data in cookies, that should be available only on one site.

Options

Domain

- …But if we'd like to allow subdomains like forum.site.com get a cookie, that's possible.

- When setting a cookie at site.com, we should explicitly set domain option to the root domain: domain=site.com:

```
// at site.com
// make the cookie accessible on any subdomain *.site.com:
document.cookie = "user=John; domain=site.com"

// later

// at forum.site.com
alert(document.cookie); // has cookie user=John
```

- For historical reasons, domain=.site.com (a dot before site.com) also works the same way, allowing access to the cookie from subdomains. That's an old notation, should be used if we need to support very old browsers.

Options

Expires, max-age

- By default, if a cookie doesn’t have one of these options, it disappears when the browser is closed. Such cookies are called “session cookies”
- To let cookies survive browser close, we can set either expires or max-age option.
- **expires=Tue, 19 Jan 2038 03:14:07 GMT**
- Cookie expiration date, when the browser will delete it automatically.

Options

Expires, max-age

- The date must be exactly in this format, in GMT timezone. We can use date.toUTCString to get it. For instance, we can set the cookie to expire in 1 day:

```
// +1 day from now
let date = new Date(Date.now() + 86400e3);
date = date.toUTCString();
document.cookie = "user=John; expires=" + date;
```

- If we set expires to a date in the past, the cookie is deleted.

Options

Expires, max-age

- **max-age=3600**
- An alternative to expires, specifies the cookie expiration in seconds from the current moment.
- If zero or negative, then the cookie is deleted:

```
// cookie will die +1 hour from now
document.cookie = "user=John; max-age=3600";

// delete cookie (let it expire right now)
document.cookie = "user=John; max-age=0";
```

Options

Secure

- The cookie should be transferred only over HTTPS.
- **By default, if we set a cookie at http://site.com, then it also appears at https://site.com and vice versa.**
- That is, cookies are domain-based, they do not distinguish between the protocols.
- With this option, if a cookie is set by https://site.com, then it doesn't appear when the same site is accessed by HTTP, as http://site.com.
- So if a cookie has sensitive content that should never be sent over unencrypted HTTP, then the flag is the right thing.

```
document.cookie = "user=John; secure";
```

Options

httpOnly

- This option has nothing to do with JavaScript, but we have to mention it for completeness.
- The web-server uses Set-Cookie header to set a cookie. And it may set the httpOnly option.
- This option forbids any JavaScript access to the cookie. We can’t see such cookie or manipulate it using document.cookie.

Options

httpOnly

- That's used as a precaution measure, to protect from certain attacks when a hacker injects his own JavaScript code into a page and waits for a user to visit that page.
- That shouldn't be possible at all, a hacker should not be able to inject their code into our site, but there may be bugs that let hackers do it.
- Normally, if such thing happens, and a user visits a web-page with hacker's JavaScript code, then that code executes and gains access to document.cookie with user cookies containing authentication information. That's bad.
- But if a cookie is httpOnly, then document.cookie doesn't see it, so it is protected.

Example Cookie Functions

- Here’s a small set of functions to work with cookies, more convenient than a manual modification of document.cookie.
- There exist many cookie libraries for that, so these are for demo purposes.

Example Cookie Functions -

- The shortest way to access cookie is to use a regular expression.

- The function getCookie(name) returns the cookie with the given name:

```
// returns the cookie with the given name,
// or undefined if not found
function getCookie(name) {
  let matches = document.cookie.match(new RegExp(
    "(?:^|; )" + name.replace(/([\.$?*|{}\(\)\[\]\\\/\+^])/g, '\\$1') + "=([^;]*)"
  ));
  return matches ? decodeURIComponent(matches[1]) : undefined;
}
```

- Here new RegExp is generated dynamically, to match ; name=<value>.

- Please note that a cookie value is encoded,
  so getCookie uses a built-in decodeURIComponent function to decode it.

Example Cookie Functions

- To delete a cookie, we can call it with a negative expiration date:

```
function deleteCookie(name) {
  setCookie(name, "", {
    'max-age': -1
  })
}
```

- Please note: when we update or delete a cookie, we should use exactly the same path and domain options as when we set it.

LocalStorage & SessionStorage

Ulster University

LocalStorage

- Web storage objects localStorage and sessionStorage allow to save key/value pairs in the browser.
- What's interesting about them is that the data survives a page refresh (for sessionStorage) and even a full browser restart (for localStorage).
- We'll see that very soon.

LocalStorage

- We already have cookies. Why additional objects?
  - Unlike cookies, web storage objects are not sent to server with each request. Because of that, we can store much more. Most browsers allow at least 2 megabytes of data (or more) and have settings to configure that.
  - Also unlike cookies, the server can’t manipulate storage objects via HTTP headers. Everything’s done in JavaScript.
  - The storage is bound to the origin (domain/protocol/port triplet). That is, different protocols or subdomains infer different storage objects, they can’t access data from each other.

LocalStorage

- Both storage objects provide same methods and properties:
  - setItem(key, value) – store key/value pair.
  - getItem(key) – get the value by key.
  - removeItem(key) – remove the key with its value.
  - clear() – delete everything.
  - key(index) – get the key on a given position.
  - length – the number of stored items.

- As you can see, it’s like a Map collection (setItem/getItem/removeItem), but also allows access by index with key(index).

LocalStorage

- The main features of localStorage are:
  - Shared between all tabs and windows from the same origin.
  - The data does not expire. It remains after the browser restart and even OS reboot.
- For instance, if you run this code…

```
localStorage.setItem('test', 1);
```

- …And close/open the browser or just open the same page in a different window, then you can get it like this:

```
alert( localStorage.getItem('test') ); // 1
```

- We only have to be on the same origin (domain/port/protocol), the url path can be different.
- The localStorage is shared between all windows with the same origin, so if we set the data in one window, the change becomes visible in another one.

LocalStorage

Looping

- As we've seen, the methods provide "get/set/remove by key" functionality. But how to get all saved values or keys?

- Unfortunately, storage objects are not iterable.

- One way is to loop over them as over an array:

```
for(let i=0; i<localStorage.length; i++) {
  let key = localStorage.key(i);
  alert(`${key}: ${localStorage.getItem(key)}`);
}
```

LocalStorage

Looping

- Another way is to use for key in localStorage loop, just as we do with regular objects.

- It iterates over keys, but also outputs few built-in fields that we don’t need:

```
// bad try
for(let key in localStorage) {
  alert(key); // shows getItem, setItem and other built-in stuff
}
```

LocalStorage

Looping

- …So we need either to filter fields from the prototype with hasOwnProperty check:

```
for(let key in localStorage) {
  if (!localStorage.hasOwnProperty(key)) {
    continue; // skip keys like "setItem", "getItem" etc
  }
  alert(`${key}: ${localStorage.getItem(key)}`);
}
```

- …Or just get the “own” keys with Object.keys and then loop over them if needed:

```
let keys = Object.keys(localStorage);
for(let key of keys) {
  alert(`${key}: ${localStorage.getItem(key)}`);
}
```

LocalStorage

Strings Only

- Please note that both key and value must be strings.

- If were any other type, like a number, or an object, it gets converted to string automatically:

```
sessionStorage.user = {name: "John"};
alert(sessionStorage.user); // [object Object]
```

- We can use JSON to store objects though:

```
sessionStorage.user = JSON.stringify({name: "John"});

// sometime later
let user = JSON.parse( sessionStorage.user );
alert( user.name ); // John
```

SessionStorage

- The sessionStorage object is used much less often than localStorage.
- Properties and methods are the same, but it’s much more limited:
- The sessionStorage exists only within the current browser tab.
  - Another tab with the same page will have a different storage.
  - But it is shared between iframes in the same tab (assuming they come from the same origin).
- The data survives page refresh, but not closing/opening the tab.

SessionStorage

- Run this code…

```
sessionStorage.setItem('test', 1);
```

- …Then refresh the page. Now you can still get the data:

```
alert( sessionStorage.getItem('test') ); // after refresh: 1
```

- …But if you open the same page in another tab, and try again there, the code above returns null, meaning “nothing found”.

- That’s exactly because sessionStorage is bound not only to the origin, but also to the browser tab. For that reason, sessionStorage is used sparingly.

Summary

- Web storage objects localStorage and sessionStorage allow to store key/value in the browser.
  - Both key and value must be strings.
  - The limit is 5mb+, depends on the browser.
  - They do not expire.
  - The data is bound to the origin (domain/port/protocol).